Mateo Usuga Vargas

Universidad Eafit

Análisis y diseño de algoritmos

Primer trabajo práctico (15%)

Fuerza Bruta

1. Descripción teórica

En un conjunto de n enteros positivos distintos, queremos encontrar todas las permutaciones posibles que cumplan con la siguiente condición:

$P[i] \leq 2 \times P[i+1]$ para todo $i \in \{0, 1, \ldots, n-2\}$

En resumen, ningún elemento puede ser más del doble que el elemente que tiene delante.

2. ¿Qué es una permutación?

Citando a la página MathWorld, "Una permutación de un conjunto de n elementos es todo arreglo ordenado posible de dichos elementos. Formalmente, una permutación es una función biyectiva de un conjunto hacia sí mismo, lo que significa que cada elemento aparece exactamente una vez en cada arreglo" (Weisstein, s.f.). Por ejemplo, para {1, 3, 4} existen 3! = 6 permutaciones:

1 3 4 / 1 4 3 / 3 1 4 / 3 4 1 / 4 1 3 / 4 3 1.

3. Enfoque de solución

Como lo hemos visto en clase, fuerza bruta se trata de pasar una por una por todas las soluciones posibles del espacio de búsqueda y revisarlas con las condiciones del problema.

En este problema el enfoque se aplica así:

1. Generar las n! permutaciones del conjunto
2. Por cada permutación, verificar si cumple la restricción
3. Imprimir las que sí la cumplen

4.¿Por qué es fuerza bruta y no backtracking?

En este ejercicio exploramos todas las posibilidades, sin descartarlas a pesar de que no cumplan con la condición, por lo tanto, no estamos aplicando ningún tipo de **poda** que nos disminuya el espacio de búsqueda, principal diferenciador del backtracking.

5. Pseudocódigo

Decidí guiarme con el pseudocódigo dado en el enunciado, igualmente, adjunto captura de pantalla:

Pseudocódigo

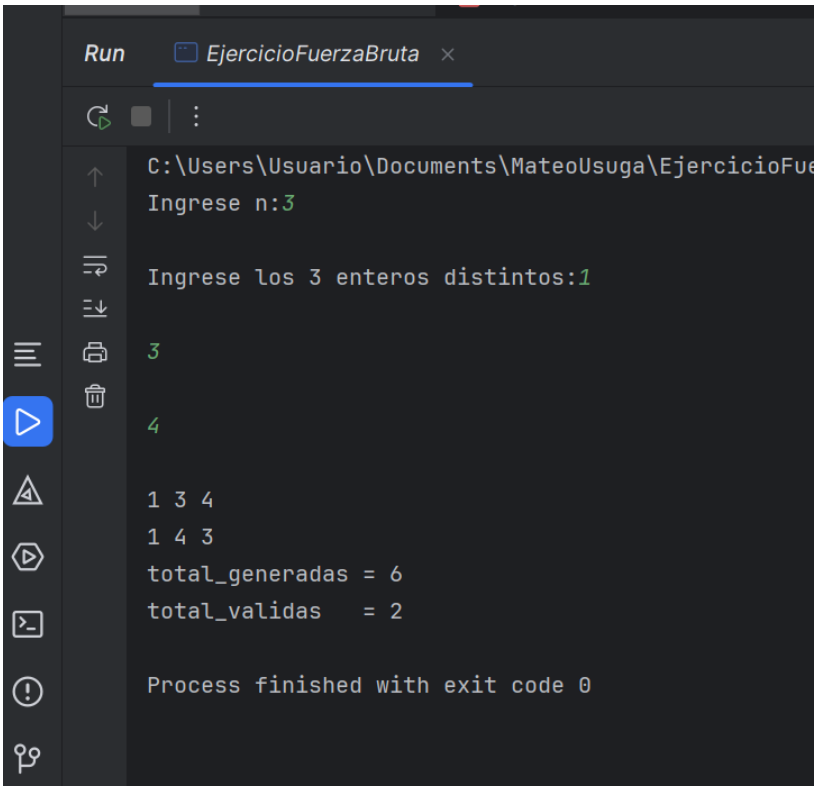
```
leer n
leer A[0..n-1]    // conjunto de enteros distintos
ordenar A asc     // garantiza comenzar desde la permutación menor

total_generadas = 0
total_validas   = 0

hacer:
    total_generadas += 1
    valida = verdadero
    para i en [0..n-2]:
        si A[i] > 2 * A[i+1]:
            valida = falso
            break
    si valida:
        imprimir A
        total_validas += 1
mientras next_permutation(A) exista

imprimir "total_generadas =", total_generadas
imprimir "total_validas   =", total_validas
```

6. Implementación en C++

(Puede encontrarse en el GitHub que acompaña esta entrega)

7. Ejemplos de entrada y salida

Al igual que en el ejemplo dado, al insertar los valores 1, 3 y 4, generamos 6 soluciones en total y 2 de ellas son válidas



Para un ejemplo más elaborado, asigne 4 a n y los valores fueron: 2,3, 9 y 15, con esos números se generaron 24 soluciones, pero tan solo 4 fueron válidas

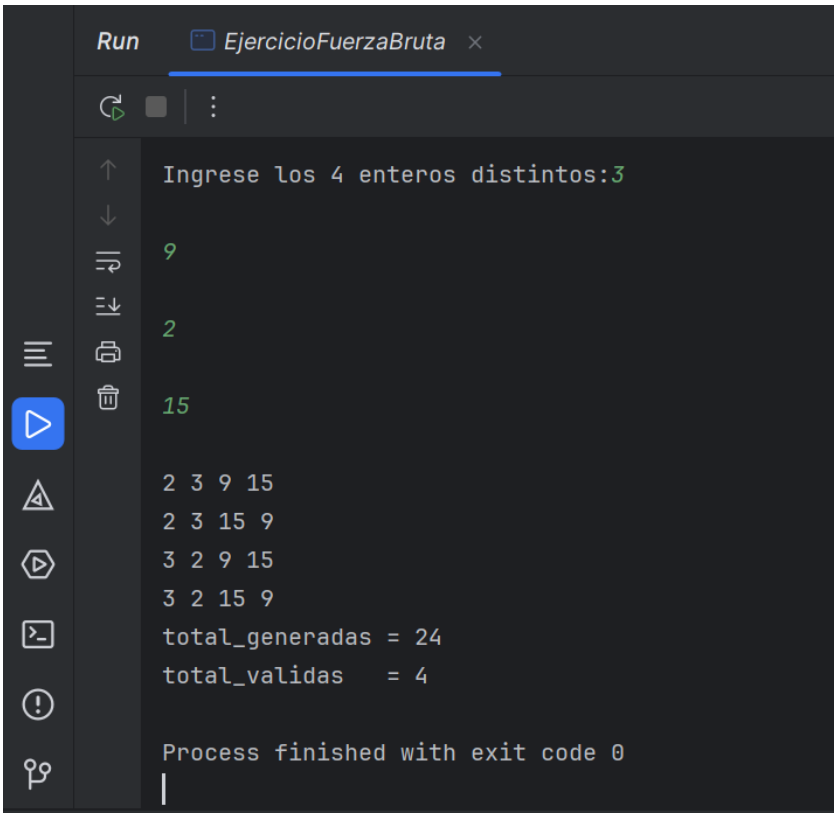
2 3 9 15

2 3 15 9

3 2 9 15

3 2 15 9

Todos cumplen con la condición asignada.



8. análisis de complejidad

Complejidad temporal

El algoritmo tiene dos partes, generación y verificación.

En generación, next_permutation genera n! Permutaciones y en verificación cada permutación se recorre con un for hasta n-1 pasos, por lo tanto, la complejidad temporal es: O(n! × n)

n	n!	n! * n	Tiempo
7			Con valores random hasta este punto es instantáneo
10	3,628,800	36,288,000	En este punto deja de ser instantáneo y se toma aprox 1 seg
12	479,001,600	5,748,019,200	Con valores random: 33.88 seg
13	6,227,020,800	81,151,270,400	Con valores random después de 7 min, seguía sin dar resultados

En conclusión, a partir de **n = 13** el tiempo se vuelve impracticable en hardware típico.

Complejidad espacial
El algoritmo solo necesita el arreglo A de tamaño n y dos contadores enteros, así que la complejidad sería O(n), no se almacenan todas las permutaciones en memoria, se imprimen y descartan

9. Preguntas guía

Proporción de válidas respecto al total al crecer n: La proporción disminuye mientras mas alto sea n. Con más elementos hay más pares que verificar en cada permutación y la probabilidad de que todos cumplan la condición se reduce.

n	Total	Válidas	Proporción
3	6	2	33%
5	120	4	3.3%
7	5040	21	0.42%
9	362880	108	0.03%

Aplicación de backtracking con poda temprana: En vez de generar permutaciones completas y evaluarlas al final, con backtracking iría evaluando la restricción conforme se fueran agregado valores de esta forma:

- Se coloca un elemento en la posición i.
- Se verifica inmediatamente si $P[i-1] \leq 2 \cdot P[i]$.
- Si la condición ya falla, se abandona toda esa rama sin explorar las $(n-i-1)!$ extensiones posibles.

next_permutation genera exactamente n! permutaciones, aunque el arreglo empieza ordenado

"Next_permutation recorre las permutaciones en orden lexicográfico estricto, avanzando una posición cada vez sin repetir ni saltarse ninguna" (cppreference.com, 2024).

Como existe exactamente una permutación entre el punto de partida y el punto final para cada reordenamiento posible, y next_permutation las recorre todas en orden sin omitir ninguna, se garantiza que el total de permutaciones procesadas es exactamente n!.

Si el arreglo no comenzara ordenado, el ciclo terminaría antes de llegar a todas las permutaciones, procesando únicamente las que están después de ese punto de partida en el orden lexicográfico.

Backtracking

1. Descripción teórica

En un grafo no dirigido G = (V, E) con n vértices y m aristas, y un número entero k que representa los colores disponibles, vamos a determinar si podemos darles un color a los vértices de tal forma que ningún par de vértices adyacentes tengan el mismo color. Esto es llamado k-coloración válida.

En resumen, dos vértices conectados no pueden tener el mismo color.

2. ¿Qué es un grafo y una k-coloración?

Según Weisstein (s.f.), "un grafo es una estructura matemática compuesta por un conjunto de vértices y un conjunto de aristas que los conectan, representado formalmente como G = (V, E) donde V es el conjunto de vértices y E el conjunto de aristas."

"Una k-coloración válida es una asignación de colores a los vértices del grafo tal que ningún par de vértices adyacentes comparta el mismo color. El número mínimo de colores necesarios para lograr una coloración válida se denomina número cromático del grafo." (Weisstein, s.f.).

3.Enfoque de solución

Igualmente, como hemos visto en clase el backtracking se basa en construir soluciones de forma que se vaya validando en cada paso para dar una poda cuando una condición no se cumpla, esto hace que evitemos revisar soluciones que de antemano sabemos que no son válidas, reduciendo el espacio de búsqueda y reduciendo en ciertos casos el tiempo de ejecución.

En este problema el enfoque se aplica así:

1. Intentar asignar un color al vértice 0
2. Avanzar al siguiente vértice y repetir
3. Si en algún vértice ningún color es válido, retroceder al vértice anterior y probar el siguiente color disponible
4. Si se colorean todos los vértices exitosamente, se cuenta como solución válida

4.¿Por qué es backtracking y no fuerza bruta?

Porque construye la solución vértice a vértice y poda cuando detecta conflicto, así evita explorar ramas que ya son inválidas y esto es lo que llamamos poda.

5.Pseudocódigo

Nuevamente decidí guiarme por el pseudocódigo proporcionado en el taller:

Pseudocódigo (backtracking)

```
leer n, k
leer matriz_ady[0..n-1][0..n-1]   // 1 si hay arista, 0 si no

color[0..n-1] = 0                  // 0 = sin color asignado
total_soluciones = 0
primera_guardada = falso

es_seguro(v, c):
    para cada u adyacente a v:
        si color[u] == c: retornar falso
    retornar verdadero

bt(v):
    si v == n:
        total_soluciones += 1
        si !primera_guardada:
            guardar color[]; primera_guardada = verdadero
        retornar
    para c en [1..k]:
        si es_seguro(v, c):
            color[v] = c
            bt(v + 1)
            color[v] = 0            // backtrack

bt(0)
si total_soluciones == 0:
    imprimir "No existe una", k, "-coloracion valida"
si no:
    imprimir "Total de", k, "-coloraciones:", total_soluciones
    imprimir "Primera solucion:", primera_solucion
```

6.Implementación en C++

(Puede encontrarse en el GitHub que acompaña esta entrega)

7.Ejemplos de entrada y salida

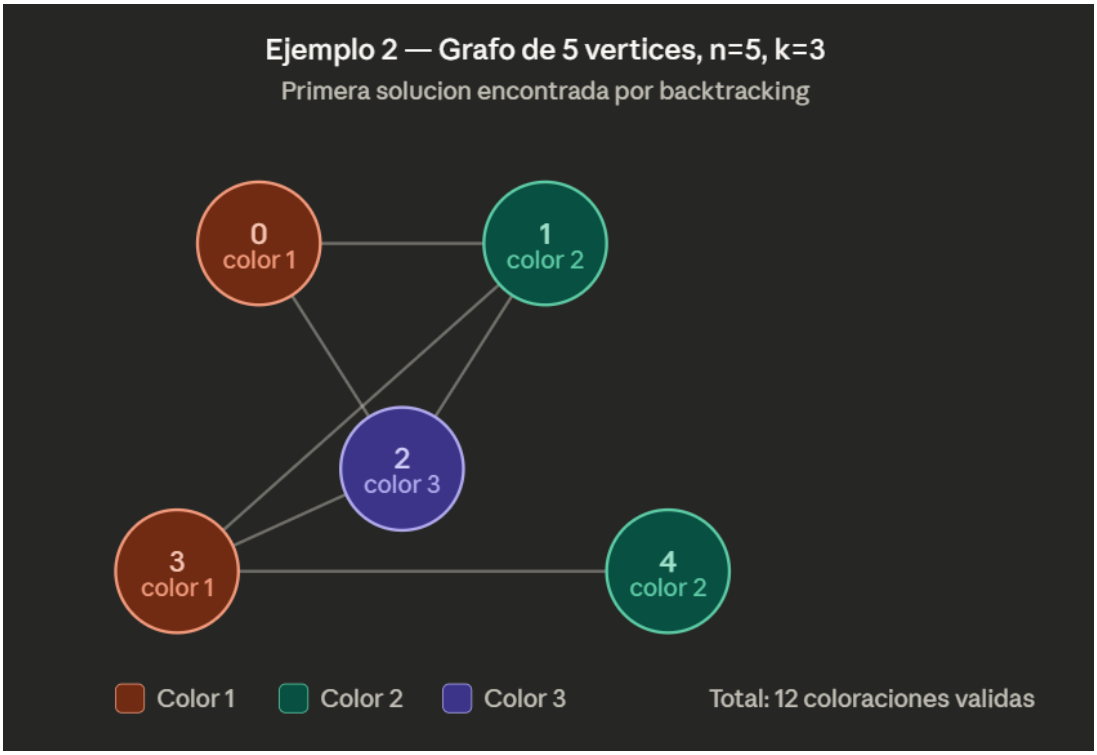
Primero revisemos el ejemplo dado en el taller, como podemos ver la salida es igual, pudimos colorear el grafo con 2 colores de los 3 disponibles en la primera solución y fuerza bruta nos confirma el total de 18 coloraciones validas.

```
C:\Users\Usuario\Documents\MateoUsuga\EjercicioBacktracking\c
Ingrese n y k:4

3

Ingrese la matriz de adyacencia:
0 1 0 1

1 0 1 0

0 1 0 1

1 0 1 0

Total de 3-coloraciones: 18
Primera solucion:
color[0] = 1
color[1] = 2
color[2] = 1
color[3] = 2
Fuerza Bruta confirma: 18

Process finished with exit code 0
```

Ahora procederé con un grafo algo más complejo para confirmar que el código funcione de manera correcta, creando 5 vértices con un máximo de 3 colores.

```
C:\Users\Usuario\Documents\MateoUsuga\EjercicioBacktracking\c
Ingrese n y k:5

3

Ingrese la matriz de adyacencia:
0 1 1 0 0

1 0 1 1 0

1 1 0 1 0

0 1 1 0 1

0 0 0 1 0

Total de 3-coloraciones: 12
Primera solucion:
color[0] = 1
color[1] = 2
color[2] = 3
color[3] = 1
color[4] = 2
Fuerza Bruta confirma: 12

Process finished with exit code 0
```

Para visualizar como sería este grafo, le he pedido a Claude que haga una representación visual y cómo podemos ver, la primera solución encontrada cumple a la perfección con la condición



8. Análisis de complejidad

Complejidad temporal

En este algoritmo usamos los dos enfoques de los que hemos estado hablando:

Backtracking: En el peor caso explora todos los nodos del árbol de búsqueda, que tiene k^n hojas. Sin embargo, gracias a la poda, en la práctica descarta grandes ramas del árbol y su rendimiento es significativamente mejor. La complejidad en el peor caso es O(k^n).

Fuerza bruta: Siempre evalúa las k^n combinaciones posibles. Para cada una recorre todos los pares de vértices, lo que añade un factor de n^2. La complejidad es O(k^n × n^2).

Grafo	n	k	Coloraciones válidas	Tiempo Backtracking	Tiempo Fuerza Bruta	Factor de mejora
Ciclo C4	4	3	18	5 µs	291 µs	~58x más rápido
Grafo 5 vértices	5	3	12	5 µs	224 µs	~45x más rápido
Grafo denso 8 vértices	8	3	6	15 µs	422 µs	~28x más rápido

Podemos evidenciar que backtracking es bastante más rapido en todas las pruebas, sin embargo, también se evidencia como al aumentar la densidad se reduce el factor de mejora, sin dejar de ser una diferencia más que considerable.

Complejidad espacial

Ambos enfoques necesitan almacenar el arreglo de colores de tamaño n y la matriz de adyacencia de tamaño n^2:

O(n^2)

9. Preguntas guía

Nodos explorados en backtracking vs fuerza bruta en el ejemplo: Fuerza bruta explora las k ^n combinaciones sin excepción, en este caso 3 ^4 = 81. El backtracking construye el árbol de búsqueda vértice a vértice y poda cuando detecta un conflicto. Para n=4,k=3, el backtracking explora unos 40 a 45 nodos, descartando la mitad de lo que fuerza bruta si exploró.

Grafo que maximiza y grafo que minimiza efecto de poda: Con un grafo muy denso con muchas aristas se maximiza la poda, porque casi cualquier asignación de colores genera conflicto, descartando muchas ramas. Por otro lado, un grafo sin aristas no nos permitiría hacer poda, ya que cualquier combinación termina siendo válida, esto hace que no se incumpla la condición y por lo tanto termina siendo lo mismo que fuerza bruta.

Cambio para saber si existe al menos una k-coloración: Se tendría que cambiar bt a una función bool que retorne true al encontrar la primera solución válida. Así, cuando esa función retorne true, cada nivel de la recursión recibirá ese true y también retornará inmediatamente, deteniendo toda la búsqueda en cadena.

Relación entre este problema y el número cromático del grafo: El número cromático χ(G) de un grafo es el mínimo valor de k para el cual existe al menos una k-coloración válida (Weisstein, s.f.). Este algoritmo se relaciona directamente con ese concepto: si se ejecuta con un valor de k y total_soluciones > 0, ese k es suficiente para colorear el grafo. Si se reduce k de uno en uno hasta que total_soluciones == 0, el valor inmediatamente anterior es exactamente el número cromático del grafo.

Webgrafía

cppreference.com. (2024). *std::next_permutation*. https://en.cppreference.com/w/cpp/algorithm/next_permutation

Weisstein, E. W. (s.f.). *Permutation*. MathWorld — A Wolfram Web Resource. https://mathworld.wolfram.com/Permutation.html

Weisstein, E. W. (s.f.). *Graph*. MathWorld — A Wolfram Web Resource. https://mathworld.wolfram.com/Graph.html

Weisstein, E. W. (s.f.). *Graph Coloring*. MathWorld — A Wolfram Web Resource. https://mathworld.wolfram.com/GraphColoring.html

Weisstein, E. W. (s.f.). *Chromatic Number*. MathWorld — A Wolfram Web Resource. https://mathworld.wolfram.com/ChromaticNumber.html

Para el desarrollo de este trabajo se utilizó Claude (Anthropic, 2025) como herramienta de apoyo en la comprensión de conceptos y revisión de código.

Por último, procedo a adjuntar algunas capturas de la conversación con Claude (Anthropic, 2025)

1. Representaciones gráficas de ciertos conceptos para mi entendimiento o para el taller

por favor construye los grafos de esta solucion y del punto del taller para ponerlos en el documento como visualizaciones

Aquí van los dos grafos con sus coloraciones tal como las encontró el programa:

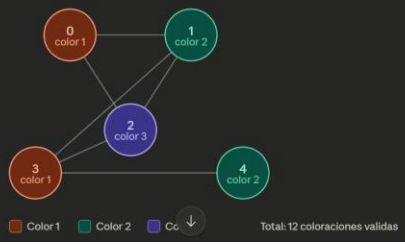
Ejemplo 1 — Ciclo C4, n=4, k=3

Primera solucion encontrada por backtracking

Y aquí el segundo ejemplo con 5 vértices, donde se necesitan los 3 colores disponibles:

Ejemplo 2 — Grafo de 5 vertices, n=5, k=3

Primera solucion encontrada por backtracking



siento que está muy abstracto y no lo visualizo bien

Entiendo, el problema es que un grafo es algo visual. Déjame mostrarte primero qué es lo que estamos resolviendo.

El problema: colorear un grafo con k colores

Ningún par de vértices conectados puede tener el mismo color

Invalido — 0 y 2 son vecinos y tienen el mismo color

Valido — ningun par vecino comparte color

Matriz de adyacencia

1 = hay arista, 0 = no hay arista

	0	1	2	3
0	0	1	1	0
1	1	0	1	1
2	1	1	0	1
3	0	1	1	0

0-1: rojo/azul ✓ 0-2: rojo/verde ✓
1-2: azul/verde ✓ 1-3: azul/azul... X ?
(1 y 3 son azul pero no son vecinos — esta bien)

Ahora que lo ves visualmente, el problema es simple:

Un grafo es un conjunto de vértices (los círculos) conectados por aristas (las líneas). El problema pregunta: ¿puedo pintar cada vértice con uno de los `k` colores disponibles de forma que ningún par de vértices conectados tenga el mismo color?

La **matriz de adyacencia** es simplemente esa misma información en forma de tabla: la fila `i` columna `j` vale `1` si hay una arista entre el vértice `i` y el `j`, y `0` si no.

Ahora veamos cómo el código resuelve esto parte por parte. ¿Quieres que arranquemos con la función `es_seguro`, el backtracking, o la fuerza bruta?

2. Explicación de ciertos conceptos

que significa cuando dices ciclo c4

Es el nombre formal de un tipo de grafo específico.

C significa **ciclo** — un grafo donde los vértices forman un círculo cerrado, es decir, cada vértice está conectado al siguiente y el último se conecta con el primero.

4 es el número de vértices.

Entonces **C4** es simplemente un ciclo de 4 vértices:

```
0 — 1
|   |
3 — 2
```

Cada vértice está conectado exactamente con sus dos vecinos, formando un cuadrado. Es exactamente el ejemplo que usa el enunciado:

```
0-1-2-3-0
```

Lo que en matriz de adyacencia se ve así:

```
0 1 0 1
1 0 1 0
0 1 0 1
1 0 1 0
```

Es solo una forma estándar de nombrar grafos conocidos. Otros ejemplos serían C3 (un triángulo), C5 (un pentágono), K4 (un grafo completo de 4 vértices donde todos están conectados con todos), etc.

3. Revisión del taller en base a las rúbricas y los elementos requeridos

PDF

perfecto, vuelve a revisar todo el documento

Revisé el documento completo. Está muy bien, solo encontré detalles menores:

Ejercicio 1 — Fuerza Bruta ✓

Todo correcto y completo. Sin observaciones.

Ejercicio 2 — Backtracking

✓ Lo que está bien

- Todas las correcciones anteriores aplicadas correctamente
- La tabla de tiempos está completa y bien explicada
- El párrafo de conclusión de la tabla es claro y correcto
- Las preguntas guía están bien respondidas
- La webgrafía está completa

⚠ Detalles menores que aún se pueden pulir

1. Página 7 — sigue diciendo "2 colores"

Escribiste:

> *"pudimos colorear el grafo con 2 colores en la primera solución"*

Esto sigue siendo impreciso. El grafo usa k=3 colores disponibles. Lo que ocurre es que la primera solución solo necesitó 2 de los 3 disponibles, pero eso no es lo mismo que decir que se colorea con 2 colores. Más preciso sería:

> *"la primera solución encontrada utilizó 2 de los 3 colores disponibles"*

---

4. Apoyo con el código, por ejemplo, con el fragmento de código que mide los microsegundos

3. Falta la tabla comparativa de tiempos entre backtracking y fuerza bruta

El enunciado lo pide explícitamente:

> *"Comparar el tiempo de ejecución de ambos enfoques en al menos 2 grafos distintos. Presentar una tabla con los resultados."*

Esto no aparece en ninguna parte del documento. Es un requisito obligatorio que puede costarte puntos en complejidad algorítmica. Necesitas correr el programa con los dos ejemplos, medir los tiempos e incluir una tabla como esta:

Grafo	n	k	Tiempo Backtracking	Tiempo Fuerza Bruta
Ciclo C4	4	3	< 1 ms	< 1 ms
Grafo 5 vértices	5	3	< 1 ms	< 1 ms

Para medir los tiempos puedes agregar esto al código temporalmente:

```cpp
#include <chrono>

auto inicio = chrono::high_resolution_clock::now();
bt(0);
auto fin = chrono::high_resolution_clock::now();
cout << "Tiempo BT: " << chrono::duration_cast<chrono::microseconds>(fin-inicio).cou
```

Resumen de correcciones

Prioridad	Corrección
Urgente	Agregar tabla de tiempos backtracking vs fuerza bruta
Importante	Corregir "2 colores" por "3 colores" y completar el número 18
Menor	Corregir la redacción de nodos explorados